

Exercises: We do All 4 Both only present the 4-6

3. Write a Python function that determines if a straight line intersects a specified circle, returning the intersection coordinates.
4. Use the function above to emulate a LiDAR shooting at a cylindrical robot as in Wasik et al. Plot the robot and the point cloud. Add noise to the measurements.
5. Place the simulated robot in front of a wall. Simulate the point cloud from both robot and wall.
6. Assign wall point cloud data to one cluster. Use the optimisation from Ex. 2 to determine the equation of the corresponding line.

Exercise 4: LiDAR point cloud on cylindrical robot

For each beam angle $\alpha_i \in [0, 2\pi]$ we check whether the ray from the LiDAR origin hits the cylindrical robot modelled as a circle at position (x_r, y_r) with radius r . The ray direction is:

$$(dx, dy) = (\cos \alpha_i, \sin \alpha_i)$$

The intersection is checked by substituting the parametric ray $P = \text{lidar_pos} + t \cdot (dx, dy)$ into the circle boundary — a point is on the circle when its distance to the center equals the radius, so we require $\|P - \text{robot_pos}\| = r$, or equivalently:

$$\|P - \text{robot_pos}\|^2 = r^2$$

This gives one equation in one unknown t — solve for t , then plug back in to get the hit point. Returns the closest hit (x_h, y_h) if $t > 0$ and within `lidar_range`, otherwise `None`.

Substituting the ray $P = \text{lidar_pos} + t \cdot (dx, dy)$ into the circle boundary condition:

$$\|P - \text{robot_pos}\|^2 = r^2 \quad \rightarrow \quad \|P - \text{robot_pos}\|^2 - r^2 = 0$$

expands to a quadratic in t :

$$\underbrace{(dx^2 + dy^2)}_{a_c} t^2 + \underbrace{2(dx \cdot x_r + dy \cdot y_r)}_{b_c} t + \underbrace{(x_r^2 + y_r^2 - r^2)}_{c_c} = 0$$

where $(fx, fy) = \text{lidar_pos} - \text{robot_pos}$. Solving with the quadratic formula:

$$t = \frac{-b_c - \sqrt{b_c^2 - 4a_c c_c}}{2a_c}$$

The smallest positive t gives the closest hit point:

$$(x_h, y_h) = \text{lidar_pos} + t \cdot (dx, dy)$$

Noise model. Each hit point is converted to polar coordinates relative to the LiDAR origin — taking the difference in x and y between hit point and lidar gives the vector between them, Pythagoras gives the true range and `atan2` gives the true bearing:

$$d = \sqrt{(x_r - x_l)^2 + (y_r - y_l)^2}, \quad \alpha = \text{atan2}(y_r - y_l, x_r - x_l)$$

where x_l, y_l is the lidar position and x_r, y_r the hit point on the robot.

Independent Gaussian noise is then added in polar domain:

$$\tilde{d} = d + \nu_d, \quad \nu_d \sim \mathcal{N}(0, \sigma_d^2) \quad \tilde{\alpha} = \alpha + \nu_\alpha, \quad \nu_\alpha \sim \mathcal{N}(0, \sigma_\alpha^2)$$

and converted back to Cartesian:

$$\tilde{x}_r = x_l + \tilde{d} \cos(\tilde{\alpha}), \quad \tilde{y}_r = y_l + \tilde{d} \sin(\tilde{\alpha})$$

Noise is added in polar domain because angle noise scales with distance — a 1° error at distance d causes a lateral shift of $d \cdot \sin(\nu_\alpha) \approx d \cdot \nu_\alpha$, while range noise σ_d is constant regardless of distance.

This is precisely the **uncertainty ellipse** — we are more certain about the range (radial axis, fixed half-length $\approx \sigma_d$) than about the lateral position (tangential axis, half-length $\approx d \cdot \sigma_\alpha$), because the same angle error σ_α produces a larger sideways displacement the further the object is — $\Delta = d \cdot \sigma_\alpha$ grows with distance while the range error stays constant.

Why angle noise is more dangerous at long range. A 1° angle error causes a lateral shift of $\Delta = d \cdot \sin(\nu_\alpha)$:

$$d = 1 \text{ m} : \quad \Delta = 1 \cdot \sin(1^\circ) \approx 1 \cdot 0.017 = 1.7 \text{ cm}$$

$$d = 10 \text{ m} : \quad \Delta = 10 \cdot \sin(1^\circ) \approx 10 \cdot 0.017 = 17 \text{ cm}$$

Range noise σ_d is constant: 5 cm error at 1 m is the same 5 cm at 10 m. So angular error scales linearly with distance — the further the object, the larger the lateral shift from the same angle mistake.

Exercise 5: Robot in front of wall

We add a wall behind the robot. For each beam angle we check the robot first — if it hits, the wall behind is occluded:

$$\text{for each } \alpha_i : \quad \begin{cases} \text{check robot} \rightarrow \text{hit} & \rightarrow \text{record robot point, skip wall} \\ \text{check robot} \rightarrow \text{miss} & \rightarrow \text{check wall} \end{cases}$$

The wall intersection uses two parametric lines — the ray and the wall segment:

$$\text{ray: } P = \text{lidar} + t \cdot (dx, dy) \quad \text{wall: } W = \text{wall_start} + s \cdot (wx, wy)$$

Setting $P = W$ and solving gives t and s via the 2D cross product:

$$\text{denom} = dx \cdot wy - dy \cdot wx$$

If $\text{denom} = 0$ the ray is parallel to the wall (cross product of two parallel vectors is zero) — no intersection possible.

Otherwise we solve:

$$t = \frac{fx \cdot wy - fy \cdot wx}{\text{denom}}, \quad s = \frac{fx \cdot dy - fy \cdot dx}{\text{denom}}$$

where $(fx, fy) = \text{wall_start} - \text{lidar}$. Two conditions must both hold for a valid hit:

- $0 < t \leq \text{lidar_range}$ — intersection is in front of lidar and within range
- $s \in [0, 1]$ — intersection lies within the actual wall segment, not on the infinite line

Concrete example: lidar at $(0, 0)$, ray going right $(dx, dy) = (1, 0)$, wall from $(5, -3)$ to $(5, 3)$ so $(wx, wy) = (0, 6)$.

Finding t — how far along the ray:

$$\text{denom} = 1 \cdot 6 - 0 \cdot 0 = 6$$

$$fx = 5, \quad fy = -3$$

$$t = \frac{5 \cdot 6 - (-3) \cdot 0}{6} = 5 \quad \rightarrow \quad \text{hit at } (0, 0) + 5 \cdot (1, 0) = (5, 0) \checkmark$$

Finding s — where on the wall segment:

$$s = \frac{5 \cdot 0 - (-3) \cdot 1}{6} = 0.5 \quad \rightarrow \quad \text{halfway along wall } \checkmark$$

If hit at $(5, -4)$: $s < 0 \rightarrow \text{miss}$. If hit at $(5, 4)$: $s > 1 \rightarrow \text{miss}$.

Exercise 6: Wall cluster and line fitting

We want to fit a line to the wall point cloud by minimising the sum of squared perpendicular distances from each point to the line $ax + by + c = 0$:

$$\min_{a,b,c} \sum_i \frac{(ax_i + by_i + c)^2}{a^2 + b^2}$$

where (a, b) is the normal vector defining the line orientation and c the offset from the origin. The denominator $a^2 + b^2$ normalises for the length of the normal vector so the result is a true perpendicular distance. The optimisation is solved numerically using `scipy.optimize.minimize`.

The normal vector (a, b) defines the orientation of the line, and c the perpendicular distance from the origin:

- $a = 1, b = 0 \rightarrow$ normal points in x -direction $\rightarrow$ line is **vertical**
- $a = 0, b = 1 \rightarrow$ normal points in y -direction $\rightarrow$ line is **horizontal**

Once a, b, c are found the line is reconstructed for plotting by solving for x from the line equation:

$$ax + by + c = 0 \quad \rightarrow \quad x = \frac{-(by + c)}{a}$$

Describes how x changes as y moves along the line — for a perfect vertical wall $b \approx 0$ so $x \approx -c/a$ is constant, confirming the line is vertical. Evaluated at $y = [y_{\min}, y_{\max}]$ of the wall points — constraining the plotted line to the actual wall segment.

Example — vertical wall at $x = 5$: $a = 1, b = 0$ because the wall is perpendicular to the x -axis, $c = -5$ because the origin is 5 units from the wall:

$$\underbrace{a \cdot x}_{\text{how far in } x} + \underbrace{b \cdot y}_{\text{how far in } y} + \underbrace{c}_{\text{offset from origin}} = 0 \quad \rightarrow \quad 1 \cdot 5 + 0 \cdot y + (-5) = 0 \checkmark$$